Cybersecurity notes

OS Command Injection

Vulnerability Notes and Techniques

A concise reference covering command execution, injection techniques, blind detection, and prevention.

Prepared by

Youssef Reda

Table of Contents

OS Command Injection **2**
- Definition . . . 2
- Impact . . . 2
- Common Causes . . . 2
- Vulnerable Example . . . 2

How Command Injection Works **3**
- Example Scenario . . . 3
- Injection Example . . . 3
- Why a Trailing Separator Can Matter . . . 3

Useful Reconnaissance Commands **4**

Blind OS Command Injection **5**
- Definition . . . 5
- Detection Methods . . . 5
- Time-Based Detection . . . 5
- Redirecting Command Output . . . 5
- Out-of-Band Application Security Testing (OAST) . . . 6
- Limitations and Impact . . . 6

Shell Metacharacters and Injection Techniques **7**
- Command Separators and Operators . . . 7
- Command Substitution . . . 7
- Quoted Context . . . 8
- In-Band vs. Blind Injection . . . 8

Shell Expansion Concepts **9**
- Environment Variables and Parameter Expansion . . . 9
- Standard Input and Text Utilities . . . 9
- PATH Security . . . 9

Preventing OS Command Injection **10**
- Preferred Defenses . . . 10
- Input Validation . . . 10
- Avoid Blacklist-Only Filtering . . . 11

Quick Reference **12**

OS Command Injection

Definition

OS Command Injection, also known as Shell Injection, occurs when an application passes untrusted input to an operating-system command unsafely, allowing an attacker to alter the intended command execution.

Impact

- **Remote Code Execution (RCE):** Execute commands with the privileges of the vulnerable process.
- **Data Exposure:** Read files or sensitive information accessible to the process.
- **Application Compromise:** Interfere with the application or its execution environment.
- **Privilege Escalation and Lateral Movement:** These may be possible when additional permissions or reachable systems permit them.

The actual impact depends on process privileges, system configuration, and reachable resources.

Common Causes

- Concatenating user-controlled input into a shell command.
- Unsafe use of functions such as PHP `system()`, `exec()`, or `shell_exec()`.
- Relying on weak filtering or blacklists of shell metacharacters.

Vulnerable Example

Consider a PHP application that constructs a command using an IP address supplied by the user:

```
system("ping " . $ip);
```

If `$ip` is not handled safely, shell syntax may allow execution beyond the intended `ping` command.

How Command Injection Works

Example Scenario

A store availability feature accepts a product ID and store ID. The application invokes a legacy script using the supplied values:

```
GET /stockStatus?productID=381&storeID=29
```

The application executes:

```
stockreport.pl 381 29
```

A vulnerable implementation may concatenate the parameters into a command:

```
system("stockreport.pl " . $productID . " " . $storeID);
```

Injection Example

An input such as the following may alter how a Unix-like shell parses the command:

```
& echo aiwefwlguh &
```

The resulting command may become:

```
stockreport.pl & echo aiwefwlguh & 29
```

The shell interprets the separators as control syntax, so the injected `echo` can execute separately from the intended script. Output such as `aiwefwlguh` is evidence that the injected command ran.

Why a Trailing Separator Can Matter

Compare the following inputs:

```
& echo aiwefwlguh
```

```
& echo aiwefwlguh &
```

Without the final separator, the remaining original argument may become an argument to `echo`. With it, the shell treats the remaining text as a separate command segment. In typical Unix-like shells, `&` runs the preceding command in the background. Exact behavior depends on the shell.

Useful Reconnaissance Commands

After confirming command execution in an authorized environment, basic commands can help identify the execution context.

Purpose	Command
Current user – Linux/Windows	`whoami`
OS details – Linux	`uname -a`
OS details – Windows	`ver`
Network configuration – Linux	`ifconfig`
Network configuration – Windows	`ipconfig /all`
Network connections and ports	`netstat -an`
Running processes – Linux	`ps -ef`
Running processes – Windows	`tasklist`

Blind OS Command Injection

Definition

Blind command injection occurs when a command executes but its output is not returned in the HTTP response. Detection therefore depends on indirect evidence rather than direct output.

Detection Methods

- **Time-based detection:** Cause a measurable delay and compare response times.
- **Output redirection:** Write output to a file and retrieve it later, if accessible.
- **Out-of-Band (OAST):** Trigger an external DNS or HTTP interaction that can be observed by the tester.

Time-Based Detection

Example Linux payload:

```
& ping -c 10 127.0.0.1 &
```

The `-c 10` option asks `ping` to send ten packets; it does not directly specify a ten-second delay. On many Linux systems this takes roughly ten seconds, but actual timing varies. A consistent response delay close to the expected duration can indicate that the injected command executed.

Redirecting Command Output

Output redirection writes command output to a file rather than the HTTP response.

```
command > output.txt 2>&1
```

- `>` redirects standard output.
- `2>&1` sends standard error to the same destination.

Whether the file can be retrieved depends on its path, permissions, and server configuration. The **web root** is the directory from which the web server serves files. Common examples include:

- Apache on Linux: `/var/www/html/`
- IIS on Windows: `C:\inetpub\wwwroot\`

These are examples only; actual locations vary. Files in the web root may be publicly accessible depending on the configuration.

Out-of-Band Application Security Testing (OAST)

OAST detects interactions between the target and an external server controlled by the tester. It is useful when command output is hidden and file-based retrieval is unavailable.

```
Injected input
     |
     v
Vulnerable application
     |
     v
Command executes
     |
     v
Outbound DNS/HTTP request
     |
     v
Tester observes interaction
```

DNS is commonly used because it is lightweight and often permitted by network rules. However, DNS traffic can be blocked, proxied, or cached. An observed interaction is evidence of an outbound lookup or request; it does not automatically prove that every stage of a larger exploit succeeded.

Limitations and Impact

Blind variants may hide command output, errors, and exit status. Detection depends on:

- Timing differences.
- Application behavior.
- Accessible output files.
- External network interactions.

Impact depends on process privileges, filesystem permissions, outbound network restrictions, and the data or systems reachable from the application.

Shell Metacharacters and Injection Techniques

Command Separators and Operators

Shell metacharacters are interpreted as syntax rather than ordinary text. Common operators include:

Operator	Typical behavior
`&`	In Unix-like shells, runs the preceding command in the background; in Windows `cmd.exe`, separates commands.
`&&`	Runs the second command only if the first succeeds.
`||`	Runs the second command only if the first fails.
`|`	Pipes standard output from the first command to standard input of the second.
`;`	In common Unix-like shells, separates commands that run sequentially regardless of the preceding command's success.
Newline	Can terminate one command and start another. URL-encoded line feed is `%0A`.

Operator support and behavior depend on the operating system, shell, and application context.

Command Substitution

In Unix-like shells, command substitution runs a command and replaces the expression with its standard output.

```
\`command\`
$(command)
```

Example:

```
echo $(whoami)
```

The shell runs `whoami` and passes its output to `echo`.

Quoted Context

Input may be placed inside single or double quotes:

```
"User Input"
'User Input'
```

When assessing a possible injection:

- Identify how the application builds the command.
- Determine whether the input is inside a quoted string.
- Check whether the input must escape that context before shell syntax can be interpreted.

Single and double quotes behave differently, and exact rules vary by shell.

In-Band vs. Blind Injection

- **In-band:** Command output is visible in the HTTP response.
- **Blind:** Output is hidden; detection uses timing, OAST, behavior changes, or accessible output files.

Shell Expansion Concepts

Environment Variables and Parameter Expansion

Environment variables store information used by processes and shell scripts.

Variable	Purpose
`PATH`	Directories searched for executable programs.
`HOME`	The user's home directory.
`USER`	Username, when defined.
`PWD`	Current working directory, when defined.

Parameter expansion substitutes a variable reference with its value before command execution:

```
echo "$HOME"
```

Standard Input and Text Utilities

Standard input (`stdin`) is a stream from which a process reads data. Many Unix utilities can process data from standard input, allowing commands to pass information between processes. Text utilities commonly perform tasks such as:

- Converting letter case.
- Reversing text.
- Encoding and decoding data.
- Searching and filtering content.

PATH Security

The `PATH` variable lists directories searched for executable programs. An unsafe configuration can cause an unintended executable to run when a command is invoked by name. Applications should use trusted execution paths and avoid relying on attacker-influenced environment settings.

Preventing OS Command Injection

Preferred Defenses

- **Avoid OS commands:** Use native platform APIs or libraries to perform the required task.
- **Avoid shell invocation:** If an external executable is necessary, use process APIs that pass the executable and each argument separately.
- **Validate input:** Apply strict allowlist validation based on the expected type and format.
- **Use least privilege:** Run the application with only the permissions it needs.
- **Restrict access:** Limit filesystem permissions and unnecessary outbound network access.
- **Monitor execution:** Log and investigate unexpected processes and outbound connections.

Input Validation

Validation should match the intended data.

- **Allowlist:** Accept only predefined permitted values when the input represents a fixed choice.
- **Numeric validation:** Require a valid numeric value when a number is expected.
- **Alphanumeric validation:** Restrict to letters and numbers only when the application's requirements allow it.

Example of allowed operation values:

```
ping
status
version
```

Valid numeric input:

```
12345
```

Invalid numeric input:

```
123; command
```

Avoid Blacklist-Only Filtering

Removing or escaping selected metacharacters is not a reliable standalone defense. Alternative encodings, shell-specific syntax, and missed characters may bypass incomplete filters. Prefer avoiding shell interpretation and using safe APIs. Validation and escaping are supplementary controls, not substitutes for safe command construction.

Quick Reference

Topic	Remember
Command injection	Untrusted input changes how an OS command is parsed or executed.
Impact	Depends on the privileges and resources available to the application process.
Metacharacters	Separators, pipes, substitutions, and quoting can alter shell interpretation.
In-band injection	Output is visible in the HTTP response.
Blind injection	Output is hidden; use timing, OAST, behavior changes, or accessible files to detect it.
OAST	An observed external interaction is evidence of a request or lookup, not proof of every exploit stage.
Blacklists	Fragile when used as the main defense.
Best defense	Avoid shell commands; otherwise pass arguments separately and validate input.
Least privilege	Limits the damage if command execution is compromised.